

MNC Patrol Effort

Technical Guide

Patrol trajectory analysis, effort summaries, coverage mapping, and dashboard reporting for Mara North Conservancy

Generated August 21, 2026

Workflow id: **mnc_patrol_effort**

1. Overview

The **mnc_patrol_effort** workflow fetches all events and linked patrol observations from EarthRanger for a specified time window. It converts patrol observations into relocations and trajectories, splits them by transport type (foot, vehicle, motorbike), and produces per-type effort summaries, coverage-grid maps, an overall conservancy occupancy calculation, and a results dashboard.

The workflow delivers:

- **total_events_recorded_by_date.csv / total_events_recorded_by_type.csv** — event counts for the period
- **total_events_recorded.html/.png** — daily events line chart
- **patrol_events.csv** — flattened patrol_info event details
- **patrol_purpose_summary.csv** — patrol count by patrol purpose
- **patrol_relocations.geoparquet** — full observation dataset with patrol metadata
- **foot_patrol_efforts.csv / vehicle_patrol_efforts.csv / motorbike_patrol_efforts.csv** — per-type effort summaries
- **foot_patrol_map.html/.png / vehicle_patrol_map.html/.png / motor_patrol_map.html/.png** — per-type coverage-grid maps
- **patrol_trajectories.geoparquet** — reprojected overall coverage grid (foot + vehicle + motorbike combined)
- **overall_patrol_efforts.csv** — per-ranger summary
- **overall_patrol_map.html/.png** — combined 1 000 m grid-cell visit density map
- **patrol_coverage.csv** — patrol occupancy percentage per conservancy region
- A **results dashboard** assembling the four maps, the events chart, and the three summary tables

Output summary

Output file	Description
total_events_recorded_by_date.csv	Daily event counts (all non-excluded types)
total_events_recorded_by_type.csv	Daily event counts broken down by event type
total_events_recorded.html / .png	Line chart of daily event totals
patrol_events.csv	Flattened patrol_info event details
patrol_purpose_summary.csv	Patrol count grouped by patrol purpose
patrol_relocations.geoparquet	All patrol observations with full patrol metadata
foot_patrol_efforts.csv	Foot patrol metrics: count, distance, duration, average speed
foot_patrol_map.html / .png	Foot patrol coverage-grid map
vehicle_patrol_efforts.csv	Vehicle patrol metrics: count, distance, duration, average speed
vehicle_patrol_map.html / .png	Vehicle patrol coverage-grid map
motorbike_patrol_efforts.csv	Motorbike patrol metrics: count, distance, duration, average speed
motor_patrol_map.html / .png	Motorbike patrol coverage-grid map

Output file	Description
patrol_trajectories.geoparquet	Reprojected overall coverage grid (foot + vehicle + motorbike)
overall_patrol_efforts.csv	Per-ranger patrol count, distance km, duration hrs
overall_patrol_map.html / .png	Combined grid-cell visit density map (RdYIGn, equal-interval 5 bins)
patrol_coverage.csv	Patrol occupancy percentage per conservancy region

2. Dependencies

2.1 Requirements (spec.yaml)

Package	Version	Channel
ecoscope-platform	>=2.15.0, <2.16.0	repo.prefix.dev/ecoscope-workflows
ecoscope-workflows-ext-custom	0.1.0rc14.*	repo.prefix.dev/ecoscope-workflows-custom
ecoscope-workflows-ext-ste	0.0.0rc1.*	repo.prefix.dev/ecoscope-workflows-custom
ecoscope-workflows-ext-wwf-virunga	0.0.0rc9.*	repo.prefix.dev/ecoscope-workflows-custom
ecoscope-workflows-ext-big-life	1.0.1.*	repo.prefix.dev/ecoscope-workflows-custom
ecoscope-workflows-ext-mnc	1.0.2.*	repo.prefix.dev/ecoscope-workflows-custom
pydeck	0.9.2	conda-forge
opentelemetry-sdk	>=1.20.0, <2.0.0	conda-forge

Note: ecoscope-workflows-ext-wwf-virunga is declared as a requirement but none of its tasks are referenced in spec.yaml.

2.2 Connections

Connection	Task	Purpose
EarthRanger	set_er_connection	Fetch event records and patrol observations; passed to get_events, get_patrol_values, and get_patrol_observations_from_patrols_df.
Dropbox (HTTP)	fetch_and_persist_file	Download the MNC community conservancy boundary gpkg and MNC parcels gpkg. Downloads are skipped if the file already exists (overwrite_existing: false).

2.3 Grouper

The workflow uses an **empty grouper list** (groupers: []). All data are processed as a single undivided dataset. The grouper is threaded through the temporal-index steps and the dashboard only — it produces no fan-out branching.

2.4 Skip conditions

task-instance-defaults.skipif applies two conditions to every task instance in the workflow by default: **any_is_empty_df** (skip if any upstream DataFrame input is empty) and **any_dependency_skipped** (skip if any upstream task was itself skipped, propagating the skip state through dependent branches).

3. Geospatial Asset Pipeline

Before any event data is fetched, the workflow downloads and prepares the two GeoPackage boundary files used as background layers on every map, and precomputes the map view state shared by all of them.

Step	Task / id	Detail
1	fetch_and_persist_file persist_mnc_gpkg	Download mnc_conservancy.gpkg from Dropbox. overwrite_existing: false — skipped on subsequent runs if the file is already present.
2	fetch_and_persist_file download_mnc_parcel	Download mnc_across_the_river_parcel.gpkg from Dropbox.
3	load_df load_comm_shp	Load the conservancy GeoPackage into a GeoDataFrame.
4	ecoscope_workflows_ext_mnc.tasks.transformation. fix_invalid_geometries fix_comm_geom	Repair invalid geometries (e.g. self-intersecting polygons) using shapely's make_valid().
5	filter_df filter_conservancy_boundary	Filter to rows where grazing_zone = 'Conservancy'. Used as the area-of-interest (AOI) for every coverage grid and for the occupancy calculation.
6	filter_df filter_mara_north	Filter to rows where name = 'Mara North Conservancy'. Used only to compute the shared map view state.
7	load_df load_mnc_parcel	Load the parcels GeoPackage.
8	ecoscope_workflows_ext_custom.tasks.results. create_geojson_layer create_conservancy_layer / create_parcel_layer	Build the two static map layers reused as background on every patrol/coverage map: a grey conservancy boundary outline and a dark-khaki parcels fill.
9	ecoscope_workflows_ext_state.tasks.spatial_operations. envelope_gdf zoom_to_envelope	Compute the bounding envelope of filter_mara_north.
10	ecoscope_workflows_ext_state.tasks.spatial_operations. compute_view_state_from_gdf_gdf_image_extent	Compute a zoom level and centre point from that envelope (pitch: 0, bearing: 0, max_zoom: 15). Reused unchanged for every draw_map call in the workflow.

4. Events Summary

A single **get_events** call (id: **get_events_data**) fetches all event types for the analysis period, with columns `id`, `time`, `event_type`, `event_category`, `reported_by`, `serial_number`, `geometry`, `created_at`, `event_details`, and `patrols`. `include_details`: true, `raise_on_empty`: true, `force_point_geometry`: true. The result feeds two branches: this events summary, and the `patrol_info` / `observations` pipeline in Sections 5–6.

4.1 Date extraction, temporal indexing, and exclusion

Step	Task / id	Detail
1	<code>extract_column_as_type</code> <code>extract_event_date</code>	Extract the time column as output_type: date into a new date column.
2	<code>add_temporal_index</code> <code>events_temporal</code>	Add a temporal index keyed on date , using the empty grouper list. <code>cast_to_datetime</code> : true, <code>format</code> : mixed.
3	<code>exclude_row_values</code> <code>filter_events</code>	Remove rows where <code>event_type</code> is any of: <code>distancecountwildlife_rep</code> , <code>distancecountpatrol_rep</code> , <code>airstrip_operations</code> .

4.2 Summaries, chart, and widget

Output	Task / id	Logic
<code>total_events_recorded_by_date.csv</code>	<code>summarize_df</code> <code>total_events_recorded</code> → <code>persist_df</code> <code>persist_tevents_df</code>	Group by date , count unique id → no_of_events . Persist as CSV.
<code>total_events_recorded_by_type.csv</code>	<code>summarize_df</code> <code>total_events_type_recorded</code> → <code>persist_df</code> <code>persist_summary_event_type</code>	Group by date and event_type , count unique id . Persist as CSV.
<code>total_events_recorded.html</code> / <code>.png</code>	<code>draw_line_chart</code> <code>draw_events_chart</code> → <code>persist_text</code> <code>persist_total_events</code> → <code>html_to_png</code> <code>convert_events_chart_png</code>	Line chart: <code>x = date</code> , <code>y = no_of_events</code> , colour <code>lightsteelblue</code> , no legend. <code>width 1280</code> / <code>height 720</code> , <code>device_scale_factor 2.0</code> , <code>wait_for_timeout 10 ms</code> .
Dashboard widget	<code>create_plot_widget_single_view</code> <code>events_chart_widget</code>	Title: 'Total Events Recorded'. Wraps the persisted chart HTML for the results dashboard.

5. Patrol Purpose Summary

This branch isolates **patrol_info** events from **events_temporal**, flattens their event details, and produces a per-purpose summary table and dashboard widget.

Step	Task / id	Detail
1	filter_df filter_patrol_info_events	Keep rows where event_type = 'patrol_info'.
2	process_events_details process_patrol_events	Flatten event details. map_to_titles: true, ordered: true.
3	normalize_json_column normalize_patrols	Normalize the event_details JSON column (skip_if_not_exists: true, sort_columns: true).
4	drop_column_prefix drop_patrol_prefix	Drop the event_details__ prefix from column names (duplicate_strategy: keep_original).
5	persist_df persist_events	Write the flattened patrol events to patrol_events.csv .
6	map_columns rename_patrol_info	Rename columns: patrols → patrol_id, Participants → participants, Patrol Purpose → patrol_purpose, Transport Type → transport_type.
7	summarize_df patrol_info_summary	Group by patrol_purpose . number_of_patrols = nunique(id), 0 decimal places.
8	persist_df persist_patrol_df	Write to patrol_purpose_summary.csv .
9	draw_table patrol_summary_table_html → persist_text patrol_summary_table_url → create_table_widget_single_view patrol_summary_table_widget	Render patrol_info_summary as an HTML table (sorting and filtering enabled, download disabled), persist it, and wrap it in a dashboard widget titled 'Patrol Purpose Summary'.

6. Patrol Observations Pipeline

Starting from **rename_patrol_info**, this pipeline expands, fetches, and merges patrol observation points from EarthRanger with patrol metadata, and converts the result into relocations for trajectory building.

Step	Task / id	Detail
1	filter_notna filter_null_patrols	Remove rows where patrol_id is null.
2	fill_missing_values fill_transport_type	Fill null values in transport_type with 'Undefined'.
3	explode explode_patrol_id	Explode the patrol_id column (reset_index: true; a row may reference multiple patrol IDs).
4	get_patrol_values get_patrol_event_values	Fetch patrol records from EarthRanger for each patrol ID (max_workers: 10).
5	get_patrol_observations_from_patrols_df get_patrol_obs	Fetch patrol observation points. include_patrol_details: true, raise_on_empty: true, sub_page_size: 100.
6	map_columns join_patrol_df	From explode_patrol_id, retain id, patrol_id, participants, patrol_purpose, transport_type.
7	merge_two_dataframes merge_patrol_df_obs	Merge get_patrol_obs (left) with join_patrol_df (right) on patrol_id .
8	explode explode_participants	Explode the participants column (reset_index: true).
9	process_relocations obs_relocs	Convert the merged observations to relocations, retaining extra__id, extra__created_at, extra__subject_id, geometry, groupby_col, fixtime, junk_status, patrol_id, patrol_title, patrol_serial_number, patrol_start_time, patrol_end_time, patrol_type, patrol_status, patrol_subject, patrol_type__value, participants, patrol_purpose, and transport_type. Filters sentinel coordinates: (180, 90), (0, 0), (1, 1).
10	persist_df persist_relocs	Write to patrol_relocations.geoparquet .

7. Trajectory Conversion

The relocations are split into three transport-type branches on the **transport_type** column. Each branch converts its relocations to trajectory segments using type-appropriate speed and distance filters, then adds a temporal index and renames columns.

7.1 Transport-type filtering

Branch	Task / id	Filter
Foot	filter_df filter_foot_patrols	transport_type = 'Foot'
Vehicle	filter_df filter_vehicle_patrols	transport_type = 'Vehicle'
Motorbike	filter_df filter_motor_patrols	transport_type = 'Motorbike'

7.2 Trajectory segment filters (relocations_to_trajectory)

Parameter	Foot	Vehicle	Motorbike
min_length_meters	0.001	0.35	0.35
max_length_meters	5 000	5 000	5 000
min_time_secs	1	1	1
max_time_secs	14 400	18 000	18 000
min_speed_kmhr	0.5	10.0	10.0
max_speed_kmhr	9.0	100.0	100.0

Note: Foot patrols use a tighter speed envelope (0.5–9 km/h) and shorter maximum duration (4 h vs 5 h) than vehicle and motorbike patrols, which use identical filter values to each other.

7.3 Temporal indexing and column renaming

Each branch (foot_trajs / vehicle_trajs / motor_trajs) then goes through two identical steps:

Step	Task	Detail
1	add_temporal_index temporal_foot_traj / temporal_vehicle_traj / temporal_motor_traj	Add a temporal index on segment_start . cast_to_datetime: true, format: mixed.

Step	Task	Detail
2	map_columns rename_foot_trajs / rename_vehicle_trajs / rename_motor_trajs	Rename extra__* columns to clean names: created_at, id, participants, patrol_end_time, patrol_id, patrol_purpose, patrol_serial_number, patrol_start_time, patrol_status, patrol_subject, patrol_title, patrol_type, patrol_type_value, subject_id, transport_type. raise_if_not_found: true.

8. Per-Type Effort Summaries

Each of the three transport-type branches is summarised identically by **patrol_type_value** and persisted as CSV — no colormap or geometry filtering happens at this stage; that is deferred to the coverage-grid pipeline in Section 9.

Branch	Task / id	Detail
Foot	summarize_df foot_patrol_metrics → persist_df persist_foot_df	no_of_patrols = nunique(patrol_id); distance_km = sum(dist_meters) m→km; duration_hrs = sum(timespan_seconds) s→h; average_speed = mean(speed_kmphr). Write to foot_patrol_efforts.csv.
Vehicle	summarize_df vehicle_patrol_metrics → persist_df persist_vehicle_df	Same aggregations. Write to vehicle_patrol_efforts.csv.
Motorbike	summarize_df motor_patrol_metrics → persist_df persist_motor_df	Same aggregations. Write to motorbike_patrol_efforts.csv.

9. Coverage-Grid Map Pipeline

Each of the three renamed trajectory branches — and, after concatenation, the combined dataset — goes through an identical grid, classification, colormap, and map-rendering pipeline. The foot branch is described here; vehicle, motorbike, and the combined ('overall') branch are identical except for task ids and output titles.

Step	Task / id (foot example)	Detail
1	create_patrol_coverage_grid foot_patrol_coverage	Overlay trajectories onto a 1 000 m grid clipped to filter_conservancy_boundary. Each cell records unique_patrol_count, time_spent_seconds/hours, and distance_patrolled_meters/km. keep_empty_cells: false.
2	reproject_gdf reproject_foot	Reproject the grid to EPSG:4326.
3	apply_classification apply_foot_class_grid	Equal-interval classification of unique_patrol_count, k = 5 bins, output column density_bins.
4	apply_color_map apply_foot_grid_colormap	Apply the RdYlGn colormap to density_bins, writing density_colors.
5	create_geojson_layer generate_foot_grid_layer	Build the DeckGL grid layer (opacity 0.55, fill from density_colors). Legend: title 'Grid Cell Visits', label_column density_bins, color_column density_colors.
6	combine_deckgl_map_layers combine_foot_patrol	Combine the grid layer with the shared parcels and conservancy boundary static layers.
7	draw_map draw_foot_map	Render using the ArcGIS hillshade/boundary tile layers, view state from gdf_image_extent, max_zoom: 10, legend placement bottom-right.
8	persist_text persist_foot_urls	Write to foot_patrol_map.html.
9	html_to_png convert_foot_png	Render to PNG. full_page: false, device_scale_factor: 2.0, wait_for_timeout: 40 000 ms, max_concurrent_pages: 1.
10	create_map_widget_single _view foot_map_widget	Title 'Foot Patrol Coverage Map'. Wraps the persisted map HTML for the results dashboard.

Note: The 40 000 ms wait_for_timeout on every map's html_to_png step gives deck.gl time to finish loading basemap tiles before the screenshot is taken; the events chart, which has no tiles to load, uses 10 ms instead.

10. Combined Trajectories and Overall Patrol Effort

After the three renamed trajectory branches complete, they are concatenated and run through the same coverage-grid pipeline as Section 9 to produce the combined ('overall') map, and separately summarised per ranger.

10.1 Concatenation and overall coverage grid

Step	Task / id	Detail
1	concatenate_dataframes concat_dataframes	Concatenate rename_foot_trajs, rename_vehicle_trajs, and rename_motor_trajs. axis: 0, join: outer, ignore_index: true, sort: false.
2	create_patrol_coverage_grid overall_patrol_coverage → reproject_gdf reproject_overall	Same grid/reprojection as each per-type branch, applied to the concatenated dataset. Feeds the classification/colormap/map pipeline that produces overall_patrol_map.html / .png and the 'Overall Patrol Coverage Map' widget (ov_map_widget).
3	persist_df persist_trajectories_data	Write reproject_overall — the reprojected overall coverage grid, not the raw concatenated trajectories — to patrol_trajectories.geoparquet .

Note: persist_trajectories_data persists reproject_overall (the grid), not concat_dataframes (the raw trajectories) — despite the filename, patrol_trajectories.geoparquet contains grid-cell coverage statistics, not per-segment trajectory geometries.

10.2 Overall patrol effort (per-ranger summary)

Step	Task / id	Detail
1	summarize_df ranger_patrol_metrics	Group concat_dataframes by participants . number_of_patrols = nunique(patrol_id) [0 d.p.]; distance_km = sum(dist_meters) m→km [2 d.p.]; duration_hours = sum(timespan_seconds) s→h [2 d.p.].
2	fill_missing_values fill_participants	Fill null participants with 'Undefined'.
3	convert_columns_to_int no_of_patrols_int	Convert number_of_patrols to int (errors: coerce, fill_value: 0).
4	persist_df persist_total_df	Write to overall_patrol_efforts.csv .

Step	Task / id	Detail
5	draw_table patrol_efforts_table_html → persist_text patrol_efforts_table_url → create_table_widget_single_view patrol_efforts_table_widget	Render, persist, and wrap as a dashboard widget titled 'Overall Patrol Efforts'.

11. Conservancy Occupancy

This section computes what fraction of the conservancy boundary was actually patrolled, using the overall (combined) coverage grid.

Step	Task / id	Detail
1	reproject_gdf reproject_conservancy	Reproject filter_conservancy_boundary to EPSG:3857 (metres).
2	ecoscope_workflows_ext _mnc.tasks.io. compute_ patrol_occupancy compute_ e_cons_occupancy	For each conservancy region, intersect its area with the unioned overall_patrol_coverage geometry. Both inputs must share a projected, metre-based CRS. Returns conservancy_name, conservancy_area_sqkm, patrolled_area_sqkm, and occupancy_percentage (rounded to 2 d.p.) per region.
3	persist_df persist_occupancy_df	Write to patrol_coverage.csv .
4	draw_table occupancy_table_html → persist_text occupancy_table_url → c reate_table_widget_singl e_view occupancy_table_widget	Render, persist, and wrap as a dashboard widget titled 'Conservancy Patrol Occupancy'.

Note: compute_patrol_occupancy raises a ValueError if either input GeoDataFrame lacks a CRS or is in a geographic (degree-based) CRS — both conservancies and patrol_coverage must already be reprojected to a projected, metre-based CRS before this task runs.

12. Results Dashboard

The final step, **gather_dashboard** (id: **mnc_events_dashboard**), assembles the workflow details, time range, groupers, and every widget built in the sections above into a single dashboard. Widgets are included in this order:

#	Widget id	Type	Title
1	foot_map_widget	Map	Foot Patrol Coverage Map
2	vh_map_widget	Map	Vehicle Patrol Coverage Map
3	mr_map_widget	Map	Motorbike Patrol Coverage Map
4	ov_map_widget	Map	Overall Patrol Coverage Map
5	events_chart_widget	Chart	Total Events Recorded
6	patrol_summary_table_widget	Table	Patrol Purpose Summary
7	patrol_efforts_table_widget	Table	Overall Patrol Efforts
8	occupancy_table_widget	Table	Conservancy Patrol Occupancy

Every map and chart widget takes a path to precomputed HTML (created via `persist_text` on the corresponding `draw_map` / `draw_line_chart` output); every table widget takes a path to HTML rendered by `draw_table` from the relevant summary dataframe.

13. Output Files

All files are written to `$ECOSCOPE_WORKFLOWS_RESULTS/`.

13.1 Events summary

File	Format	Description
total_events_recorded_by_date.csv	CSV	Daily event counts (all non-excluded types)
total_events_recorded_by_type.csv	CSV	Daily event counts by event_type
total_events_recorded.html	HTML	Interactive line chart of daily event totals
total_events_recorded.png	PNG	Static version of the events chart (2× DPI)

13.2 Patrol purpose

File	Format	Description
patrol_events.csv	CSV	Flattened patrol_info event details
patrol_purpose_summary.csv	CSV	Patrol count by patrol purpose
patrol_purpose_summary_table.html	HTML	Rendered table backing the dashboard widget

13.3 Relocations

File	Format	Description
patrol_relocations.geoparquet	GeoParquet	Full observation dataset: all patrol types, with patrol metadata columns (patrol_id, patrol_title, patrol_type, patrol_purpose, transport_type, participants, etc.)

13.4 Per-type patrol effort and coverage

File	Format	Description
foot_patrol_efforts.csv	CSV	Foot: patrol count, distance km, duration hrs, average speed
foot_patrol_map.html / .png	HTML / PNG	Foot patrol coverage-grid map (RdYIGn)
vehicle_patrol_efforts.csv	CSV	Vehicle: patrol count, distance km, duration hrs, average speed
vehicle_patrol_map.html / .png	HTML / PNG	Vehicle patrol coverage-grid map (RdYIGn)
motorbike_patrol_efforts.csv	CSV	Motorbike: patrol count, distance km, duration hrs, average speed
motor_patrol_map.html / .png	HTML / PNG	Motorbike patrol coverage-grid map (RdYIGn)

13.5 Combined trajectories, overall effort, and coverage

File	Format	Description
patrol_trajectories.geoparquet	GeoParquet	Reprojected overall coverage grid (foot + vehicle + motorbike combined)
overall_patrol_efforts.csv	CSV	Per-ranger patrol count, distance km, duration hrs; nulls replaced with 'Undefined'
overall_patrol_efforts_table.html	HTML	Rendered table backing the dashboard widget
overall_patrol_map.html / .png	HTML / PNG	Combined 1 000 m grid-cell visit density map (RdYIGn)
patrol_coverage.csv	CSV	Patrol occupancy percentage per conservancy region (2 decimal places)
patrol_coverage_table.html	HTML	Rendered table backing the dashboard widget

14. Software Versions

Duplicated from Section 2.1 for reference — see spec.yaml for the authoritative, current requirement pins.

Package	Version constraint	Channel
ecoscope-platform	>=2.15.0, <2.16.0	https://repo.prefix.dev/ecoscope-workflows/
ecoscope-workflows-ext-custom	0.1.0rc14.*	https://repo.prefix.dev/ecoscope-workflows-custom/
ecoscope-workflows-ext-ste	0.0.0rc1.*	https://repo.prefix.dev/ecoscope-workflows-custom/
ecoscope-workflows-ext-wwf-virunga	0.0.0rc9.*	https://repo.prefix.dev/ecoscope-workflows-custom/
ecoscope-workflows-ext-big-life	1.0.1.*	https://repo.prefix.dev/ecoscope-workflows-custom/
ecoscope-workflows-ext-mnc	1.0.2.*	https://repo.prefix.dev/ecoscope-workflows-custom/
pydeck	0.9.2	conda-forge
opentelemetry-sdk	>=1.20.0, <2.0.0	conda-forge